Lab 5: Static Code Analysis

Library Fine and Book Renewal System (`library_system.py`)

Name	Sai Likith S
SRN	PES1UG24AM239
Assigned repo	github.com/SETAPESU26/L5_01_library_fines
Submission repo	github.com/sailikith5/SE_AM_239 (folder `Lab-5`)
Tools	Pylint 4.1.3, Bandit 1.9.4, Flake8 7.4.1 (Python 3.12)

1. Objective and approach

The goal was to analyse the assigned Python program with Pylint (code quality), Bandit (security) and Flake8 (PEP 8 style), document the issues, and fix them. I first ran the program and read its output, then ran the three tools in a virtual environment and saved the reports to `reports/before`. I fixed the problems in three commits, one per tool, then re-ran every tool and saved the results to `reports/after`.

2. Results summary

Tool	Before	After
Pylint	6.07 / 10 (22 messages)	10.00 / 10 (target was above 9.5)
Bandit	2 issues: 1 Medium (B307 eval), 1 Low (B110 try/except/pass)	0 issues
Flake8	14 messages (E302, E305, E501, E722, F401)	0 messages

3. Program behaviour before fixing

Running the original program showed problems that the tools do not report. These were found by reading the output:

• The count *loan dates on record* went 1, 2, 3 for different students, because of a shared mutable default list.
• *Fine for 5 days* printed 15 instead of 10, because `renewBook` with 21 extra days raised the global fine rate from 2 to 3.
• *Fine for -3 days* printed -9, a negative fine.
• A loan with the integer book id 4521 was accepted, although the source comment says ids must be strings, and a loan with an empty student id was dropped without any message.
• Returning a book that was never borrowed produced no message at all.
• Typing text instead of a number for the days crashed the program with a traceback.

4. Issues found and how they were fixed

Issue	Type	Line(s)	Description	Fix approach
eval() on rule string (Bandit B307, Pylint W0123)	Security (Medium)	45	fine_from_rule passed a caller-supplied string to eval, allowing arbitrary code execution.	Replaced with an ast-based evaluator that only allows numbers, days, + - * / and unary +/-; anything else raises ValueError.
Bare except: pass (Bandit B110, Pylint W0702, Flake8 E722)	Security / Bug (Low)	21-22	return_book swallowed every error, including typos and Ctrl-C, so failed returns were silent.	Catch only (KeyError, ValueError) and print a clear "no such loan" message.
Mutable default argument (Pylint W0102)	Bug	9, 12	dates=[] is shared across calls, so "loan dates on record" kept growing (1, 2, 3) for unrelated students.	Default to None and create a new list inside the function.
Renewal changes the global fine rate (not flagged by tools; Pylint W0603 on global)	Logic bug	27-31	renewBook with more than 14 extra days did FINE_PER_DAY += 1, so renewing a book raised fines for everyone (5 days cost 15 instead of 10).	Removed the global mutation, so renewal only reports the renewal. Also renamed to renew_book (Pylint C0103).

Issue	Type	Line(s)	Description	Fix approach
Negative fine (not flagged by tools)	Logic bug	24-25	calculate_fine(-3) returned -9, i.e. the library would owe the student money.	Clamp with max(days_overdue, 0).
Integer book id accepted (not flagged by tools)	Bug	56	add_book_loan("S103", 4521) stored a non-string id, which the source comment says is wrong. Empty student ids were also dropped silently.	Validate that book_id is a non-empty string and print a message for both invalid cases.
Unclosed files, no encoding (Pylint R1732, W1514)	Quality / Bug	33-42	open() without with leaks the handle if an exception occurs; encoding left to platform default.	Use with open(..., encoding="utf-8"); load_records also handles missing or corrupt JSON.
global loans reassignment (Pylint W0603)	Design	35	load_records rebound the module global.	loans.clear() then loans.update(data).
Unhandled int(days) input	Bug	63	Typing text such as abc crashed the program with a traceback.	Wrap in try/except ValueError with a friendly message.
main() runs on import	Design	68	Importing the module executed the demo and prompted for input.	Added if __name__ == "__main__": guard.
Unused import os (Pylint W0611, Flake8 F401)	Style	2	Dead import.	Removed.
Missing docstrings (Pylint C0114/C0116)	Style	1 and all functions	No module or function documentation.	Added concise docstrings.
Old-style % / + string building (Pylint C0209)	Style	14, 31	String formatting could be f-strings.	Converted to f-strings.
Blank lines and long lines (Flake8 E302, E305, E501)	Style	9-68	Missing blank lines between functions, two lines over 79 chars.	Reformatted to PEP 8.

5. Key code changes

Bandit B307: eval() replaced by a safe evaluator

```
BEFORE
def fine_from_rule(rule, days):
    return eval(rule.replace("days", str(days)))

AFTER (only numbers, the name "days" and + - * / are allowed)
def fine_from_rule(rule, days):
    return _eval_node(ast.parse(rule, mode="eval"), days)
```

Bandit B110 / Pylint W0702: bare except with pass

```
BEFORE                                  AFTER
except:                                 except (KeyError, ValueError):
    pass                                    print(f"Cannot return {book_id}: "
                                                f"{student_id} has no such loan")
```

Pylint W0102: mutable default argument

```
BEFORE: def add_book_loan(student_id, book_id, dates=[]):
AFTER:  def add_book_loan(student_id, book_id, dates=None):
            if dates is None:
                dates = []
```

Logic bugs: renewal changing the fine rate, and negative fines

```
BEFORE
def renewBook(student_id, book_id, extra_days):
    global FINE_PER_DAY
    if extra_days > 14:
        FINE_PER_DAY = FINE_PER_DAY + 1
def calculate_fine(days_overdue):
    return days_overdue * FINE_PER_DAY

AFTER
def renew_book(student_id, book_id, extra_days):
    print(f"Renewed {book_id} for {student_id} by {extra_days} days")
def calculate_fine(days_overdue):
    return max(days_overdue, 0) * FINE_PER_DAY
```

Pylint R1732 / W1514 / W0603: file handling and global state

```
AFTER
def load_records(file="loans.json"):
    try:
        with open(file, "r", encoding="utf-8") as handle:
            data = json.load(handle)
    except (FileNotFoundError, json.JSONDecodeError) as err:
        print(f"Could not load records from {file}: {err}")
        return
    loans.clear()
    loans.update(data)
```

6. Verification (after the fixes)

Final tool output on the fixed file:

```
pylint library_system.py  ->  Your code has been rated at 10.00/10
```

Program output after the fixes (Enter pressed for the days prompt):

```
S101 borrowed B-ALGO (loan dates on record: 1)
S102 borrowed B-NETW (loan dates on record: 1)
Cannot record loan of B-DBMS: missing student id
Cannot record loan for S103: book id must be a non-empty string
Cannot return B-NONE: S999 has no such loan
Renewed B-ALGO for S101 by 21 days
Fine for 5 days: 10
Fine for -3 days: 0
Fine via rule: 18
S102 owes: 0
Loan Report
S101 -> ['B-ALGO']
S102 -> ['B-NETW']
```

7. Commits

#	Commit message	Tool addressed
1	Fix Bandit findings: replace eval() with safe AST evaluator (B307), handle exceptions instead of try/except/pass (B110)	Bandit
2	Fix Pylint findings: docstrings, mutable default arg, global state, with-open, f-strings, naming; fix logic bugs (fine rate mutation, negative fines, int book id)	Pylint
3	Fix Flake8 findings: PEP 8 blank lines and line length	Flake8
4	Add before/after static analysis reports, issues.md and reflection.md	Deliverables

Deliverables in `Lab-5/`: `library_system.py`, `issues.md`, `reflection.md`, and `reports/before` and `reports/after` (pylint_report.txt, bandit_report.txt, flake8_report.txt).